

Parcial Programación III

Sistema de Gestión de Reservas de Biblioteca

REVISAR QUÉ ARCHIVOS SE ENTREGAN. SI LA ENTREGA NO SE REALIZA DE MANERA CORRECTA LA NOTA FINAL ES 1 (UNO).

Una biblioteca universitaria desea informatizar su sistema de préstamos y reservas de libros. La administración necesita poder:

Registrar libros y sus categorías

Registrar usuarios de la biblioteca

Registrar préstamos de libros Consultar información sobre préstamos y disponibilidad.

REQUISITOS FUNCIONALES (100 puntos)

RF01 – CRUD de Libros (15 puntos)

Implementar alta, baja, modificación y consulta de libros. Cada libro tiene:

- Título
- Autor
- Categoría
- CantidadDisponible

RF02 – Registrar un nuevo préstamo (25 puntos)

Validaciones: El usuario debe existir .El libro debe existir. Debe haber al menos 1 unidad disponible. La fecha de devolución debe ser posterior a la fecha actual.

Si algo falla, lanzar **PrestamoInvalidoException**.

Si es exitoso: El estado será ACTIVO. Debe descontarse una unidad disponible del libro.

RF03 – Finalizar un préstamo (15 puntos)

Permitir finalizar un préstamo activo, cambiando su estado a FINALIZADO.

Además: Debe incrementarse nuevamente la cantidad disponible del libro

RF04 – Listar préstamos de un usuario (15 puntos)

Dado el ID de un usuario, mostrar todos sus préstamos. Debe mostrarse también: nombre del usuario, email, título del libro prestado

RF05 – Consultar libros por categoría (15 puntos)

Dada una categoría (ej: “Programación”, “Base de Datos”), mostrar todos los libros registrados de esa categoría.

■ **Este punto debe resolverse utilizando obligatoriamente Streams**

RF06 – Consultar préstamos vencidos (15 puntos)

Mostrar todos los préstamos cuya fecha de devolución ya haya pasado y cuyo estado siga siendo ACTIVO.

■ **Este punto debe resolverse utilizando: comparación de fechas con LocalDate Streams**

BASE DE DATOS

```
CREATE DATABASE biblioteca;  
USE biblioteca;
```

```
CREATE TABLE usuario (  
id INT PRIMARY KEY AUTO_INCREMENT,  
nombre_completo VARCHAR(100),  
dni VARCHAR(20),  
email VARCHAR(100) );
```

```
CREATE TABLE libro (  
id INT PRIMARY KEY AUTO_INCREMENT,  
titulo VARCHAR(100),  
autor VARCHAR(100),  
categoria VARCHAR(50),  
cantidad_disponible INT );
```

```
CREATE TABLE prestamo (  
id INT PRIMARY KEY AUTO_INCREMENT,  
id_usuario INT,  
id_libro INT,  
fecha_prestamo DATE,  
fecha_devolucion DATE,  
estado ENUM('ACTIVO', 'FINALIZADO', 'CANCELADO'),  
FOREIGN KEY (id_usuario) REFERENCES usuario(id), FOREIGN KEY (id_libro)  
REFERENCES libro(id) );
```